Recall-Aware Early Termination in Approximate Nearest Neighbor Search

Shuang Hao
School of Computer Science and Technology
Beijing Jiaotong University
Beijing, China
haoshuang@bjtu.edu.cn

Xinxin Li
School of Computer Science and Technology
Beijing Jiaotong University
Beijing, China
23125236@bjtu.edu.cn

Wei Zhang
School of Computer Science and Technology
East China Normal University
Shanghai, China
zhangwei.thu2011@gmail.com

Abstract

Approximate nearest neighbor search (ANNS) is a fundamental operation in large-scale vector retrieval systems, where achieving high recall under strict latency constraints is essential. Existing ANNS approaches typically control recall using fixed search parameters, such as a predefined candidate neighbor set (CNS) size, which require significant effort to fine-tune for each use case. Even worse, due to substantial heterogeneity in query difficulty and data distribution, static parameterization often results in over-searching for easy queries and under-searching for hard ones. In this paper, we propose an adaptive framework for ANNS that explicitly incorporates user-specified recall requirements into the search process by employing an early termination strategy. We introduce two learning-based mechanisms, one for dynamically predicting the achieved recall during the search and triggers early termination once the predicted recall satisfies the target threshold, and another for estimating the minimal CNS size required to satisfy user needs during the search, enabling dynamic scaling of the CNS. Our framework is index-agnostic and can be seamlessly integrated into widely used graph-based ANN indexes, including HNSW, with negligible overhead. Extensive experiments on benchmark datasets and various graph indexes demonstrate that our methods significantly reduce query latency compared to state-of-the-art baselines. Our code is available at https://github.com/lxxabb/Recall-Aware-Early-Termination-in-Approximate-Nearest-Neighbor-Search.

CCS Concepts

• **Information systems → Search engine architectures and scalability**.

Keywords

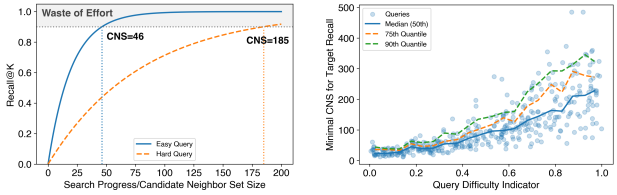
Approximate Nearest Neighbor Search, Early Termination

ACM Reference Format:

Shuang Hao, Xinxin Li, and Wei Zhang. 2026. Recall-Aware Early Termination in Approximate Nearest Neighbor Search. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770855.3817686



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
https://doi.org/10.1145/3770855.3817686



(a) Recall Growth for Different Queries (b) CNS Distribution with Quantile Bands

Figure 1: Query-dependent CNS Requirements

1 Introduction

Nearest neighbor search (NNS) is a fundamental operation in numerous applications, including information retrieval [15, 18, 23], pattern recognition [14], and recommendation systems [21]. As the scale of the datasets grows and the dimensions increase, it is necessary to scan almost all the data to retrieve the exact nearest neighbor for a query [28]. To cope with that dimension disaster, the approximate NNS (ANNS) is adopted, which provides a better tradeoff between accuracy and efficiency.

Most existing ANNS methods control the recall/latency tradeoff by tuning search parameters, such as the candidate neighbor set (CNS) size. In practice, these parameters are heavily fine-tuned to satisfy the desired recall on average for each use case. However, real-world query workloads exhibit substantial heterogeneity [4, 27, 29]. Some queries are easy and can reach high recall with minimal search effort, while others are significantly harder and require much larger search budgets. As a result, fixed parameterization often leads to over-searching for easy queries, wasting computation and increasing latency, while still failing to provide sufficient recall for hard queries. This is illustrated in Figure 1(a), which shows the growth of the recall of different queries during the search process (*i.e.,* as the size of the CNS increases). If we set the same CNS size for both queries, say 180, the hard query fail to achieve the target recall 0.90 even when the search is terminated, while the easy query has already reached 1.00 recall long before that. The time spent afterwards is wasted. This implies that if we can precisely estimate the recall of a query at any point during traversal or predict the required CNS size for each query to achieve its target recall, we can provide an efficient declarative recall solution.

Moreover, the application or the user specifies recall requirements rather than low-level search parameters. This motivates a fundamentally different design principle for ANNS, namely that the system should dynamically determine when to stop searching based on whether the desired recall has already been achieved. To

this end, Li et al. [16] first propose *AdaptNN* to adaptively determine search termination conditions for individual queries. For each query, AdaptNN first performs a fixed-budget search and estimates the remaining number of distance computations required to retrieve nearest neighbor. In practice, this prediction is scaled by a hand-tuned multiplier to ensure adequacy, resulting in additional manual tuning. Chatzakis et al. [5] introduce *DARTH* to predict current recall during traversal and terminate the search early when it meets the recall target. DARTH incorporates two critical hyperparameters to control the prediction interval, which are specified as not requiring tuning and can be obtained heuristically (by dividing the average number of distance calculations needed to reach the target on the training set by a specific constant). However, we find that this heuristic-based configuration fails to generalize across different scenarios. For example, when alternative graph indexes are preferred instead of HNSW, substantial manual tuning remains necessary to ensure high efficiency or achieve the desired recall. Therefore, designing an effective early termination strategy is nontrivial, as it must simultaneously address multiple challenges to achieve both efficiency and accuracy.

Challenge I: Appropriate Timing for Predictions. The timing of the predictions is a crucial factor in affecting efficiency, particularly for methods requiring multiple predictions. This is because the computational cost of the model inference may offset the benefits gained from early stopping. Previous approaches control the prediction timing through hyperparameters, whose values can be determined via heuristic methods or tuning approaches. Methods relying on hyperparameter tuning often suffer from limited robustness and increased deployment and maintenance costs, which hinder their practicality in real-world systems. Although heuristics offer a lightweight solution, they generalize poorly across application scenarios. In practice, it is difficult to accurately determine the optimal timing for predictions under different workloads without hyperparameters to decide.

Challenge II: Effective Features and Predictor Model. Another key challenge in learning-based early termination lies in selecting appropriate features and models. The required search depth to achieve a target recall is an implicit and highly non-linear variable, which depends on complex interactions between the query, data distribution, and index structure. Observable features only provide indirect signals of search difficulty, making it non-trivial to identify informative yet efficient representations. Moreover, the choice of model must balance prediction accuracy, inference overhead, and robustness under strict recall constraints, since inaccurate predictions may either violate recall target or negate the benefits of early termination.

To address these challenges, we propose a run-time adaptive approach, exploiting two predictors, to solving the problem of declarative recall for ANNS applications. We also design a selector to determine which one to use for each query. The first mechanism, similar to DARTH [5], utilizes a *recall predictor* to estimate the current recall during the traversal and determine whether the search can be halted once the specified recall target is satisfied. Unlike DARTH, we avoid heuristic scheduling of prediction intervals and instead derive an upper bound on the current recall. When this bound reaches the target, the predictor triggers a termination decision. As the target recall is approached, the gap between this upper bound and the true recall diminishes, thereby preventing missed opportunities for early termination at the optimal moment. Even with careful design, this mechanism inevitably requires multiple predictions. To further reduce inference overhead, we introduce a *CNS predictor* that estimates the minimal CNS size required to achieve a target recall. This design requires only a single prediction when the traversal reaches the k-th position of the CNS, where k denotes the number of nearest neighbors to be returned. Based on this prediction, we dynamically resize the CNS and allow the search to terminate naturally.

We identify the features responsible for the recall and CNS prediction, respectively, as they pose fundamentally different feature requirements. Recall prediction primarily relies on retrospective signals that summarize the already explored search space, such as the stability of top-k results and their distance distributions. In contrast, CNS prediction is a prospective task that aims to estimate the remaining search effort, and thus depends more on features that capture query difficulty and the latent potential of unexplored regions. These differences highlight that CNS estimation cannot be treated as a straightforward extension of recall prediction. Actually, estimating the remaining CNS size is fundamentally a harder task than recall prediction, as it requires safe extrapolation under partial observability and heavy-tailed uncertainty.

We opt to use the gradient boosting decision tree (GBDT) [13] for the prediction model due to its highly efficient training and inference capabilities. However, in the task of CNS prediction, the required CNS sizes exhibit a long-tailed distribution, since large values corresponding to difficult queries are relatively rare, which can be seen in Figure 1(b). GBDT trained under standard objectives tend to regress predictions toward the dense low-value region. In the context of CNS prediction, such an underestimation is particularly problematic, as it imposes a hard limit on the search space and can result in unmet recall targets. To address this issue, we formulate the CNS prediction as a quantile regression problem, allowing the model to explicitly control the risk of underestimation by targeting high quantiles of the conditional CNS distribution without introducing manually tuned safety margins.

Our extensive experiments on various benchmark datasets and graph indexes demonstrate that our method could always meet the declared recall targets on average and achieve up to 5x, 3x and 2x speedup compared to the greedy search without early termination, AdaptNN and DARTH, respectively.

2 Preliminary

2.1 The Recall-Aware ANNS Problem

The nearest neighbor search aims to find the vectors in a given set that are closest to a given query vector. With the growth of data volume and vector dimensions, approximate nearest neighbor search has emerged as an alternative solution that improves efficiency by sacrificing a small amount of accuracy to obtain the approximate nearest neighbors. Formally, the ANNS problem [24] is defined as:

Definition 1. *Let $X = \{x_1, \cdots, x_n\} \in \mathbb{R}^D$ denote a set of n vectors with D dimensions, and q represent the query. Given a value*

$k \le n$, ANNS finds the k closest vectors in X to q based on a pair-wise distance function $d\langle q, x\rangle$.

Remark. It is actually the approximate k nearest neighbor search (AKNNS) problem. But for simplicity, we use ANNS as the unified term in the following discussion unless there is ambiguity.

The quality of approximation is often measured by recall, and recall@k denotes the recall when searching for k nearest neighbors. Specifically, suppose that $\mathcal{N}_k$ denote k approximate nearest neighbors in X of the query q. When the implications are clear, we use $\mathcal{N}$ instead. And $N_k^{\checkmark}$ (or $N^{\checkmark}$) denote k true nearest neighbors of q. The recall@k is then computed as $recall@k = \frac{|\mathcal{N} \cap N^{\checkmark}|}{k}$.

In this paper, the user will provide the expected recall as input for the ANNS algorithm, which we denote by γ, and we explore the recall-aware ANNS problem, which can be formally defined as:

Definition 2. *Given a dataset X, a query q, and a target recall $\gamma \in (0, 1]$, the goal is to retrieve a set of k candidate neighbors such that at least a fraction γ of the true k nearest neighbors of q are included.*

2.2 ANNS with Graph Indexing

Indexing is a commonly used technique to improve the efficiency of the ANNS, as it reduces the number of distance calculations. State-of-the-art ANNS systems employ navigable proximity graphs, including HNSW [20] and NSG [7], to support efficient graph-based traversal in high-dimensional spaces. These graphs differ mainly in their construction and pruning strategies, but share a common greedy search paradigm [26]. Given a query q, graph-based nearest neighbor search typically starts from an entry node and performs greedy traversal on the proximity graph. At each step, the algorithm explores neighboring vertices that are closer to q, while maintaining a candidate set of visited points ordered by their distances. The search proceeds by iteratively expanding the closest unvisited candidate until a termination condition is met. In practice, it is bounded by a predefined budget, such as the maximum size of the candidate neighbour set (CNS), to control latency. The larger the CNS, the more vertices are traversed, resulting in higher recall but longer search times.

3 Methodology

In this section, we provide a detailed explanation of our recall predictor (Section 3.1) and CNS predictor (Section 3.2) and conclude by discussing how to choose between them in practical scenarios (Section 3.3).

3.1 Recall Predictor

Our first early stopping strategy is to predict the recall during the search process, which is shown in Algorithm 1 in the Appendix. Concretely, we collect features at a specified moment during the traversal and predict current recall γ_p. If $\gamma_p < \gamma$, the search will continue until the next prediction or until the nodes in the CNS no longer change. If $\gamma_p > \gamma$, we can conclude that the target is met and the search can be terminated safely.

3.1.1 The Timing of Prediction. It should be noted that the cost of feature collection and model prediction may offset the benefit gained from early termination. Thus, it is necessary to carefully consider when to make predictions, with the goal of minimizing the cost while being able to stop the search in a timely manner. Ideally, the prediction should be performed just as the recall reaches the target. However, this is almost impossible to achieve.

Table 1: Features of Recall Predictor

Type	Feature	Description
Search	n_{step}	Number of search steps
	n_{dis}	Number of distance calculations
	n_{insert}	Number of nodes inserted into the first k positions
Distance	d_0	Distance from q to the first node
	d_{k-1}	Distance from q to the kth node
	μ_d	Mean of distances from q to the first k nodes
	σ_d	Variance of distances from q to the first k nodes
CNS	r_{visited}	Ratio of visited nodes among the first k positions
	n_{stable}	The duration of the above ratio

Our approach is grounded in the following observation. When the current recall reaches the target γ, at least $\lceil k \times \gamma \rceil$ nodes in the CNS are true neighbors of q and stably occupy the top k positions. Equivalently, when the number of neighbors of the visited node inserted into the first k positions of the CNS remains below $k - \lceil k \times \gamma \rceil$ for a non-trivial duration, we say that the CNS has reached a relatively stable state and the target recall is likely to have been achieved. Here, we estimate the minimal stability duration from the training set, defined as the average stable duration μ_γ observed when the recall first reaches γ.

We determine the timing of the first prediction by exploiting the stability condition. Once a prediction is made, the model output can be used to derive an upper bound on the current recall (It is merely an estimated upper bound, as the model prediction cannot be 100% accurate). This upper bound subsequently governs future predictions. Specifically, subsequent predictions are triggered only when the search re-enters a stable state and the estimated upper bound exceeds the target recall. We now formalize the computation of this recall upper bound. Let m denote the number of newly inserted nodes that have entered the top k positions of the CNS since the last prediction. Given the previously predicted recall γ_p, we define the upper bound of the current recall as $\gamma_p + m/k$ since each newly added node in the top k positions can contribute at most $1/k$ to the recall, particularly in the later stages of the search. This upper bound provides a lightweight and principled criterion for recall-aware early termination decisions.

If the predictor consistently underestimates the recall for certain queries, the estimated upper bound may never reach the target recall, thereby preventing predictions from being triggered for these cases. To mitigate this issue, we derive from the training set the average stable duration μ_1 when the recall reaches 1.0. We further stipulated that a prediction must be made whenever the number of consecutive stable steps exceeds μ_1, regardless of whether the upper bound has reached the target recall. This mechanism acts as a safeguard against persistent underestimation.

3.1.2 Input Features. Since our dynamic recall predictor is designed to estimate recall at arbitrary stages of the search, input

features must effectively capture the current search progress. Consequently, we categorize these features into three groups (see Table 1 for details).

Search progress features. These features characterize the search progress through cumulative statistics collected from the start of the search to the time of feature extraction. They include the number n_{step} of steps taken, *i.e.,* the number of nodes visited in the CNS, the number n_{dis} of distance calculations performed, and the total number n_{insert} of nodes inserted into the first k positions of CNS, which may have since been moved out of the top k positions). Larger values for these features indicate a longer search duration, which may correspond to higher recall.

Distance features. The distance features are selected since they can indicate whether the neighbors found currently are close to the query q. These include the distance from q to the first node d_0 and the kth node d_{k-1}, as well as the mean μ_d and variance σ_d of distances from q to the first k vertices in the CNS.

CNS status features. The ratio of visited nodes among the top-k positions, denoted as r_{visited}, together with its persistence duration n_{stable}, serves as an indicator of whether the current CNS has reached a relatively stable state. A larger value of r_{visited} generally corresponds to a higher recall, while a longer persistence duration increases the reliability of this indication.

3.1.3 Recall Predictor Model. We choose gradient boosting decision trees as the predictor model. We build and train them using the LightGBM library [13] because of its excellent inference time for single-input predictions. In addition, tree-based models provide an intuitive way to assess the importance of features by aggregating the error reduction achieved at each split, which facilitates feature analysis and helps explain the behavior of the model. The recall predictor is trained on a dataset derived from observations collected during the execution of training queries. After each distance computation, we extract the features listed in Table 1 and annotate them with the corresponding recall value at that moment. This procedure enables the predictor to capture fine-grained search dynamics and to learn from observations at arbitrary stages of the search process.

3.2 CNS Predictor

Even with careful design, the aforementioned recall prediction method still requires multiple model invocations for one query, which introduces additional costs. Therefore, in this section, we try to directly predict the size L of the CNS required to achieve the accuracy desired by the user.

3.2.1 The Timing of Prediction. We cannot estimate L before the search begins, as it is unrealistic to accurately predict the required size of the CNS based solely on the query q. Thus, we collect intermediate search features after traversing the kth position of the CNS (since the CNS must be at least k), predict the minimal CNS size required to achieve the target recall, and scale down the CNS to achieve early termination.

3.2.2 Input Features. Compared to recall estimation, predicting the required CNS size is inherently more difficult, as it requires extrapolating the remaining search effort from partial and potentially intermediate observations. Unlike recall prediction, underestimating the required CNS size is unsafe, as it may lead to premature termination and failure to meet target recall, thereby imposing stricter requirements on prediction accuracy and robustness. Therefore, we employ additional features to predict the size of the CNS, which could be categorized into four groups (see Table 2 for details).

Table 2: Features of CNS Predictor

Type	Feature	Description
Search	n_{step}	Number of search steps
	n_{dis}	Number of distance calculations
	n_{insert}	Number of nodes inserted into the first k positions
	μ_{hop}	Average number of hops required to access each vertex in the CNS
Distance	d_0	Distance from q to the first node
	d_{k-1}	Distance from q to the kth node
	μ_d	Mean of distances from q to the first k nodes
	σ_d	Variance of distances from q to the first k nodes
	H_d	Entropy over the normalized distances from q to the first k nodes
	d_{en}	Distance from q to the entry node
	r_{en}	The ratio of d_{en} to the average distance from the entry node to its neighbors
Cosine	$\mu_{\cos}$	Mean of cosine similarities between q and the first k nodes
	$\sigma_{\cos}$	Variance of cosine similarities between q and the first k nodes
	$H_{\cos}$	Entropy over cosine similarities between q and the first k nodes
Degree	μ_{deg^-}	Average in-degree of vertices in the CNS

Search progress features. The first three indicators of search progress are the same as recall predictor. They provide auxiliary signals on the efficiency of the traversal. The required CNS size is positively correlated with the search difficulty of a given query q. Accordingly, we introduce μ_{hop}, the average number of graph hops required to reach the vertices in the CNS from the entry node. The larger μ_{hop} indicates that the CNS vertices are harder to access during traversal, suggesting increased search difficulty, and consequently a larger CNS is needed to identify the nearest neighbors of q.

Distance features. Distance features d_0, d_{k-1}, μ_d and σ_d are still critical for the prediction of the CNS size, since these intermediate distance statistics reflect how closely the search has approached the query, how compact the current neighborhood is, and how much uncertainty remains. In addition, this category also includes the entropy over the normalized distances from q to the first k nodes (H_d), the distance from q to the entry node, denoted by d_{en} and the ratio of d_{en} to the average distance from the entry node to its neighbors (r_{en}).

Distance entropy plays a critical role in predicting the required CNS size. A large CNS is often required not because the nearest neighbors are far away but because the search needs to disambiguate among multiple competing regions. The entropy naturally captures this uncertainty. The entry–query distance provides a prior on search difficulty and acts as an implicit indicator of whether the search starts within the query's attraction basin.

Cosine similarity features. The cosine similarity statistics between the query and the top-k candidates in the CNS reflect directional consistency, which is critical for estimating the remaining search

effort. They include the mean $\mu_{\cos}$, variance $\sigma_{\cos}$ and entropy $H_{\cos}$ of consine similarities between q and the first k nodes. Directional consistency often stabilizes earlier than distance in a high-dimensional search. As a result, cosine similarity statistics offer an early and robust signal of search convergence, which is crucial to predicting the remaining CNS size required to achieve the target recall.

Degree features. The in-degree of nodes in the CNS reflects the structural accessibility of the graph during the search. A CNS dominated by high-degree nodes typically indicates that the search is well guided by the graph structure, leading to faster convergence and a smaller required candidate set. Although in-degree is not directly query-conditioned, it provides a complementary structural signal for predicting the remaining search effort. However, note that it cannot be used on graph indexes such as HNSW where all nodes have the same degree.

In addition, we would like to clarify that although the additional features used in the CNS predictor are informative, they are not well suited for recall prediction due to differences in prediction objectives and invocation stages. The recall predictor estimates the current search progress and is invoked early and repeatedly, requiring lightweight and stable features. In contrast, the CNS predictor estimates future search effort and is invoked only after the search has sufficiently progressed. Consequently, features such as entropy, hop statistics, and degree-related measures become reliable only in later stages and may introduce noise in early-stage recall prediction.

3.2.3 CNS Predictor Model. To predict the required CNS size, we employ a GBDT model with quantile regression, implemented using the LightGBM library. GBDT models are well suited for this task as they can effectively capture complex, non-linear interactions among heterogeneous features derived from the search process such as distance statistics and directional consistency. Moreover, tree-based models naturally handle feature scaling and mixed feature types, making them robust in practice. The training data for the CNS predictor are collected by conducting searches on a set of training queries. For each training query, we extract feature snapshots after traversing the kth position of the CNS. We execute the search until it reaches the target recall using the standard termination condition, and record the minimal CNS size required to satisfy the recall constraint. This minimal CNS size serves as the ground-truth label.

A key challenge in CNS prediction lies in the long-tailed distribution of required CNS sizes. Although most queries can be resolved with a relatively small candidate set, a small fraction of hard queries require substantially larger CNS values. Under such skewed distributions, standard regression objectives based on mean squared or absolute error tend to bias predictions toward the dominant low-value region, which may lead to systematic underestimation for hard queries and consequently unsafe early termination.

To address this issue, we adopt quantile regression, which allows the model to estimate a conditional quantile of the CNS distribution given the observed features, rather than predicting its conditional mean. By directly modeling an upper quantile of the conditional CNS distribution, the predictor produces conservative yet data-driven estimates that better account for long-tail behavior. Importantly, the selected quantile does not constitute an additional tunable hyperparameter in the traditional sense. The quantile level encodes a fixed safety requirement rather than a tunable heuristic and therefore does not increase the model's hyperparameter complexity. The target quantile is specified once, and the corresponding loss function is natively supported by LightGBM.

Table 3: Dataset Description and Indexing Summary

Dataset	Dim	#Base	#Query	HNSW			NSG		
				M	*efC*	*efS*	*L*	*R*	*C*
SIFT	128	1,000,000	10,000	32	500	500	40	50	500
GIST	960	1,000,000	1,000	32	500	1000	60	70	500
DEEP	96	10,000,000	10,000	32	500	750	60	70	500
GLOVE	100	1,183,514	10,000	16	500	500	60	70	500

3.3 Selection of Predictors

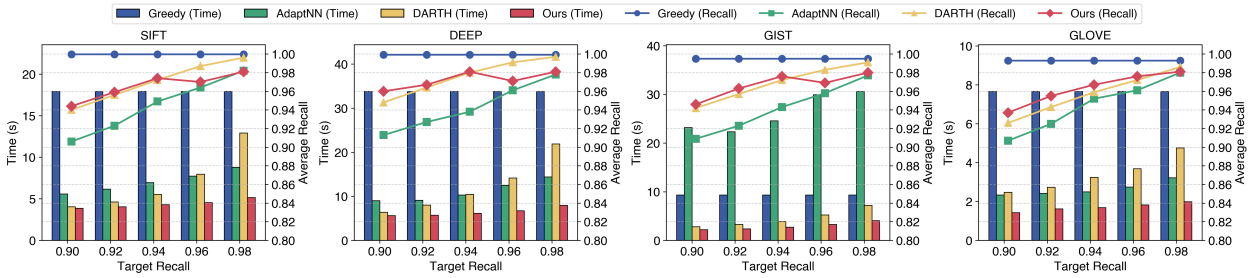
Finally, we discuss how to select an appropriate predictor in practical deployments. The choice of predictor is determined by the target recall and the search state. Since the CNS predictor is invoked only once per query and directly estimates the remaining search budget, it exhibits consistently strong performance, especially under high target recall settings (*e.g.* ≥ 0.95) or for hard queries. When the target recall is low or the query is relatively easy, the CNS prediction becomes less effective. This is because low recall targets are often satisfied early in the search, whereas the size of CNS is predicted later and with unnecessarily conservatism, which may delay termination. Therefore, it is necessary to adaptively select between the two predictors on a per-query basis.

One possible approach is to predict the appropriate predictor before the search begins using the query vector as features with a LightGBM model. Our experiments show that this approach is feasible. However, it introduces additional prediction overhead prior to the search. Therefore, this approach is particularly suitable for scenarios where queries arrive in batches, allowing the prediction cost to be amortized through batch inference.

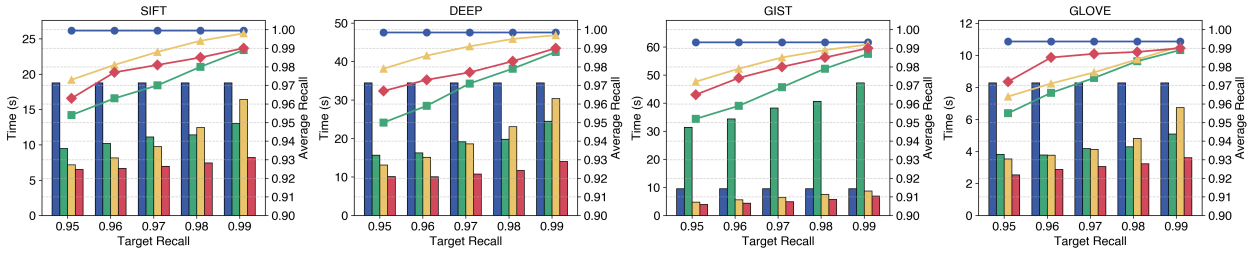
Another approach is to leverage intermediate search states for adaptive selection. Specifically, before the search reaches the k-th position of the CNS, we employ the recall predictor. Once it is reached, we determine whether to switch to the CNS predictor or continue using the recall predictor based on the current search state. For example, the CNS predictor is invoked when the distance distribution remains dispersed, indicated by high distance entropy (H_d) or large variance (σ_d), or when the average hop (μ_{hop}) is large. To learn suitable features and thresholds for the selector, we construct a training set that is identical to those used by the CNS predictor but the label is a binary indicator, which is 1 if using the CNS predictor results in lower latency and 0 otherwise. We then train a decision tree classifier and the selector is directly derived from the learned tree and implemented as a set of if–else rules. The cost of prediction is negligible in this way, so we prefer this approach.

4 Experiment

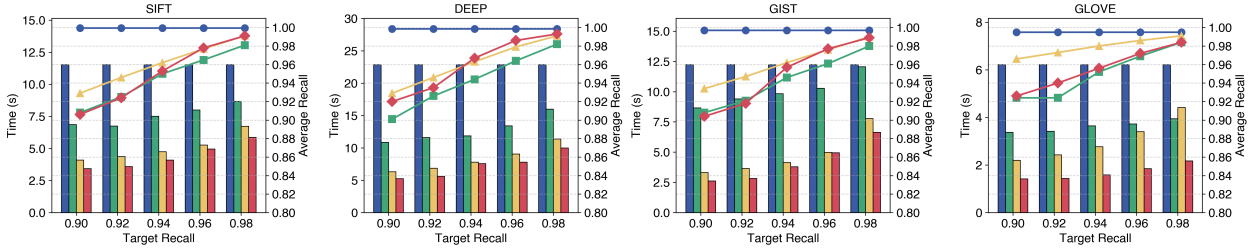
We conduct experiments to showcase the effectiveness of our method, addressing the following questions: **RQ1:** How does our method's perform compared to competitors across varied real-world datasets? **RQ2:** How many queries meet the user's recall requirements? **RQ3:**



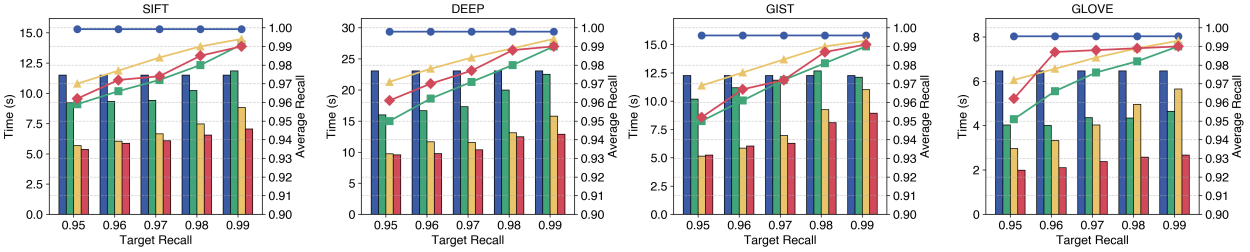
(a) HNSW, $k = 50$



(b) HNSW, $k = 100$



(c) NSG, $k = 50$



(d) NSG, $k = 100$

Figure 2: Overall Evaluation of Our Method

Is our predictors making accurate predictions? **RQ4:** How important are the features?

4.1 Experimental Settings

Datasets. We evaluate our approach on four widely used datasets in the literature: SIFT1M [11], GIST1M [11], DEEP10M [2] and GLOVE1M [1, 22]. The details are summarized in Table 3. Note that we strictly follow the official splits provided by the datasets without any modification. Both predictors are trained on the data sampled from the training set and evaluated on the held-out query vectors.

Index. For each dataset, we build the HNSW [20] and NSG [7] index once, using the appropriate parameters that allow the index to reach an average ≥ 0.99 recall for query workloads. The indexing details are also shown in Table 3.

Baselines. We compare our method with the following baselines. (1) *Greedy* [26], which is the standard greedy graph traversal method; (2) *AdaptNN* [16], which predicts the total number of distance computations required for a query at a fixed point during the search and stops the traversal once this budget is reached; (3) *DARTH* [5], which repeatedly estimates the current recall during graph traversal using a learned predictor and terminates the search once the estimated recall reaches the target.

Parameter Settings. In real-world scenarios, users often demand high recall. Thus, we use the recall target $\gamma \in \{0.90, 0.92, 0.94, 0.96, 0.98\}$ for $k = 50$ and $\gamma \in \{0.95, 0.96, 0.97, 0.98, 0.99\}$ for $k = 100$. For all baselines, we follow the original implementations and parameter settings as described in their respective papers.

Evaluation Metrics We measure the performance of each ANNS search algorithm using the *total search time* and *average recall* for the query set. We also demonstrate *the distribution of recall* achieved by each query and the *target success rate*. We measure the accuracy of our predictors using the *Mean Square Error (MSE)*, *Mean Absolute Error (MAE)*, and *R-squared (R^2)* which are usually utilized to evaluate the performance of regression models.

Experimental Environment. We conduct our experimental evaluation on a server running Ubuntu 20.04.6 LTS, with 64 GB of main memory and an Intel(R) Xeon(R) 6982P-C processor (8 cores, 3.60 GHz). Each query experiment is repeated three times, and the average result is reported to reduce system variability.

4.2 Overall Evaluation of Our Method (RQ1)

We first evaluate the total search time of our method with all baselines across different datasets and target recall under different graph indices and k values. We use a selector introduced in Section 3.3 to determine which predictor to utilize for each query. Following the previous work [5], we set a large CNS to ensure that over 99% of queries achieve the target accuracy, and the reported time includes the cost of parameter tuning if required. Note that the average recall of all methods have met the target.

The experimental results demonstrate that our method outperforms all baselines in most cases. Compared to greedy search, our approach significantly reduces redundant exploration by enabling early termination once the target recall is confidently met. As for AdaptNN, despite making only a single prediction, it requires significant parameter tuning time, particularly on high-dimensional GIST datasets. In contrast, both DARTH and our approach derive relevant parameters directly from the training data. Compared to DARTH, which also relies on recall prediction, our method benefits from fewer prediction attempts. DARTH adopts a heuristic prediction strategy, where the recall predictor is invoked multiple times during a single query, resulting in an average of 6–18 predictions per query depending on the target recall. In contrast, a single prediction by our recall predictor takes approximately 0.01 ms. Under HNSW, it performs 1–3 predictions per query (2.24 on average, accounting for 3.4% of the total time). The CNS predictor takes about 0.04 ms per prediction and is invoked only once per query, accounting for 7.3% of the total time on average. And the selector can choose a more appropriate stopping mechanism for each query. Our design ensures that the total inference overhead is minimal compared to the graph traversal time.

The Performance of Our Selector. Table 4 demonstrate the search time and average recall achieved (shown in parentheses) of the full framework and the use of the recall predictor (Φ_r) and the CNS predictor (Φ_c) alone in two representative scenarios, namely $k = 50$ with $\gamma = 0.90$ and $k = 100$ with $\gamma = 0.95$. We can see that using the full framework is the optimal approach in the vast majority of cases. This is because queries that achieve the target recall before reaching the k-th position of the CNS can be terminated early by the recall predictor, while the remaining queries are handled by an adaptively selected predictor based on the observed search state, ensuring that each query benefits from the most appropriate early-stopping strategy (The accuracy of our selector is about 0.78). Even when this adaptive strategy does not outperform the best individual predictor, the performance gap is typically small. It is also worth noting that the selector's preference varies substantially across datasets, index types, and target recall levels. These observations highlight the importance of adaptive predictor selection.

Table 4: Effectiveness of the Selector

Dataset	Φ_r	Φ_c	Ours
SIFT	**3.86s** (0.944)	4.10s (0.951)	3.88s (0.944)
GIST	3.02s (0.952)	2.38s (0.926)	**2.20s** (0.946)
DEEP	6.13s (0.952)	**5.63s** (0.952)	5.64s (0.960)
GLOVE	1.69s (0.940)	1.68s (0.969)	**1.43s** (0.937)

(a) HNSW, $k = 50$, $\gamma = 0.90$

Dataset	Φ_r	Φ_c	Ours
SIFT	7.72s (0.975)	7.79s (0.975)	**6.54s** (0.963)
GIST	5.38s (0.976)	4.76s (0.968)	**3.94s** (0.965)
DEEP	15.17s (0.981)	11.21s (0.970)	**10.12s** (0.967)
GLOVE	2.92s (0.965)	2.99s (0.985)	**2.54s** (0.972)

(b) HNSW, $k = 100$, $\gamma = 0.95$

Dataset	Φ_r	Φ_c	Ours
SIFT	4.57s (0.939)	6.07s (0.952)	**3.43s** (0.906)
GIST	4.47s (0.949)	5.23s (0.905)	**3.62s** (0.904)
DEEP	6.32s (0.953)	6.94s (0.956)	**5.25s** (0.920)
GLOVE	1.77s (0.934)	2.22s (0.975)	**1.42s** (0.926)

(c) NSG, $k = 50$, $\gamma = 0.90$

Dataset	Φ_r	Φ_c	Ours
SIFT	5.41s (0.970)	6.02s (0.971)	**5.37s** (0.962)
GIST	7.06s (0.973)	6.99s (0.956)	**5.24s** (0.952)
DEEP	13.18s (0.979)	11.06s (0.965)	**9.59s** (0.961)
GLOVE	**1.94s** (0.962)	2.28s (0.984)	1.99s (0.962)

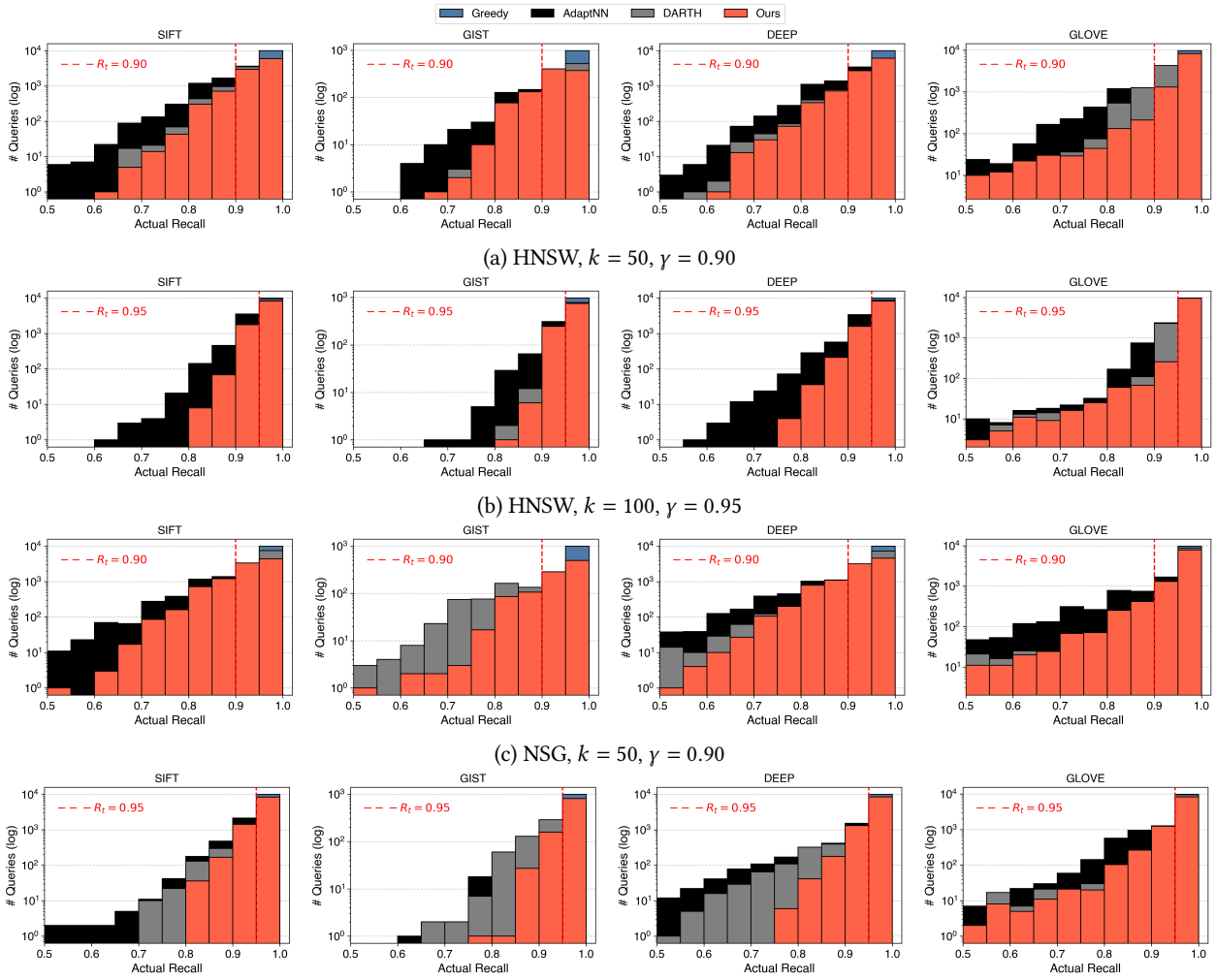
(d) NSG, $k = 100$, $\gamma = 0.95$

Table 5: Target Success Rate across All Value of k

Dataset	HNSW			NSG		
	AdaptNN	DARTH	Ours	AdaptNN	DARTH	Ours
SIFT	70.44%	90.56%	**90.94%**	71.81%	82.05%	**84.68%**
GIST	67.55%	86.38%	**88.08%**	70.25%	**83.57%**	80.69%
DEEP	68.66%	93.25%	**93.33%**	75.16%	84.72%	**90.15%**
GLOVE	76.78%	83.14%	**90.95%**	78.86%	89.73%	**90.18%**

4.3 Per-Query Performance (RQ2)

We next compare the search quality of different approaches. Figure 3 illustrates the per-query recall distributions of all methods across all datasets for $k = 50, \gamma = 0.90$ and $k = 100, \gamma = 0.95$. While all methods achieve the target recall on average, substantial differences can be observed at the per-query level. In fact, our method achieves slightly lower average recall compared to other methods in most cases, indicating that our approach stops early in a timely manner. However, the number of queries achieving target recall is comparable. For example, in the GLOVE dataset, our method achieves the recall of 0.95 for 95.36% of queries (approaching the 96.63% achieved by greedy search) under HNSW, whereas DARTH only reaches for 74.61% of all queries, and AdaptNN achieved 65.82%.



(a) HNSW, $k = 50$, $\gamma = 0.90$

(b) HNSW, $k = 100$, $\gamma = 0.95$

(c) NSG, $k = 50$, $\gamma = 0.90$

(d) NSG, $k = 100$, $\gamma = 0.95$

Figure 3: Query Recall Distribution

Table 5 reports the proportion of queries that reach the target recall for three early-stopping methods across all value of k. In fact, our method achieves the highest target success rate in most cases. Even in the few cases where it does not, the achieved average recall still satisfies the user-specified target γ. Therefore, for queries that fail to meet γ, the actual recall typically deviates only marginally from the desired target.

4.4 The Accuracy of Our Predictors (RQ3)

In this part of experiments, we evaluate the accuracy of our recall predictor (Table 6) and CNS predictor (Table 7). Note that the MSE, MAE and R^2 measures are averaged over all configurations. Table 6 shows that the recall predictor consistently achieves low prediction errors in all datasets and graph indices. The R^2 values are close to 1.0 in most cases. This indicates that the change of recall during graph-based ANNS is highly regular and can be reliably characterized by a small number of features. This also demonstrates the effectiveness of estimating the upper bound of the current recall using the recall from previous predictions.

In contrast, Table 7 demonstrates that predicting the required minimum CNS size is significantly more challenging. Despite using a richer and more diverse feature set, the CNS predictor exhibits larger absolute errors and relatively lower R^2 scores. This is expected, as the required CNS size is strongly influenced by long-tail behaviors and query-specific search difficulty, especially for hard queries that require substantially more expansions to reach the target recall. These rare but extreme cases heavily inflate regression metrics such as MSE and reduce R^2, even though the majority of predictions remain sufficiently accurate for practical decision-making.

Details of Model Training. We study the impact of the size of the training set on the error of our two predictors by varying the number of training queries from 1,000 to 20,000, across different graph indexes and different values of k. We find that increasing the amount of training data consistently reduces the prediction error for all datasets. However, once the training size exceeds 10,000 queries, the prediction error becomes nearly unchanged. This trend is consistently observed across all datasets. Thus, we randomly

Table 6: Recall Predictor Error across All Value of k

Dataset	HNSW			NSG		
	MSE	*MAE*	R^2	*MSE*	*MAE*	R^2
SIFT	0.007	0.065	0.898	0.006	0.060	0.928
GIST	0.008	0.068	0.885	0.012	0.081	0.864
DEEP	0.010	0.076	0.864	0.009	0.069	0.903
GLOVE	0.004	0.050	0.917	0.003	0.042	0.958

Table 7: CNS Predictor Error across All Value of k

Dataset	HNSW			NSG		
	MSE	*MAE*	R^2	*MSE*	*MAE*	R^2
SIFT	705	13.61	0.35	606	12.63	0.39
GIST	33940	126.74	0.29	29170	104.53	0.32
DEEP	3714	28.61	0.49	3200	26.73	0.51
GLOVE	5677	28.29	0.27	4912	26.56	0.26

Table 8: Training Time Details using 10K Queries

Dataset	Recall Predictor		CNS Predictor	
	HNSW	*NSG*	*HNSW*	*NSG*
SIFT	18s+2.4s	20s+1.17s	3s+0.35s	2s+0.25s
GIST	41s+2.8s	38s+1.78s	11s+0.34s	14s+0.35s
DEEP	26s+2.3s	30s+1.12s	11s+0.39s	13s+0.39s
GLOVE	13s+1.4s	10s+0.89s	2s+0.33s	3s+0.31s

select 10K queries to train our two predictors in all experiments. Table 8 demonstrates the time (with 8 threads) required to generate approximately 2M training data from 10K queries and the training time needed for the GBDT model (100 estimators and 0.1 learning rate). The training process completes within a few seconds, since the required training data are collected on-the-fly during query processing by recording intermediate features, recall values, and visited CNS positions. Combined with the high training efficiency of GBDT, the overall training overhead is negligible.

4.5 The Study of Feature Importance (RQ4)

In the end, we analyze the importance scores of the features, which are extracted from our GBDT predictors. Due to space limitation, the importance of different feature types is shown here. For the specific results of each feature, please refer to the Appendix.

Table 9 shows that the recall predictor is dominated by distance features across almost all datasets and graph indices, accounting for over half of the total importance. This confirms that recall estimation mainly depends on how close and concentrated the current CNS is around the query. Search progress features also contribute meaningfully, while CNS stability features play a minor role under HNSW and become more relevant under NSG, reflecting differences in traversal dynamics between the two graphs.

In contrast, Table 10 reveals that CNS prediction relies on a more balanced combination of signals. Search progress and distance features jointly dominate, while cosine-based features contribute substantially on high-dimensional datasets such as GIST, highlighting the importance of directional consistency for estimating search difficulty. Degree features have lower importance but capture complementary structural information.

Table 9: Aggregated Feature Importance of Recall Predictor

Dataset	HNSW			NSG		
	Search	*Distance*	*CNS*	*Search*	*Distance*	*CNS*
SIFT	34.63%	60.09%	5.26%	20.26%	65.19%	14.38%
GIST	30.45%	63.92%	5.60%	36.58%	46.05%	17.36%
DEEP	36.29%	57.99%	5.69%	29.56%	52.30%	18.13%
GLOVE	43.22%	51.52%	5.23%	23.73%	59.07%	17.20%

Table 10: Aggregated Feature Importance of CNS Predictor

Dataset	HNSW			NSG			
	Search	*Distance*	*Cosine*	*Search*	*Distance*	*Cosine*	*Degree*
SIFT	28.95%	48.34%	9.54%	48.76%	40.76%	3.73%	5.15%
GIST	36.05%	38.73%	24.95%	43.88%	23.31%	23.33%	3.47%
DEEP	45.56%	42.33%	6.84%	46.02%	28.81%	5.68%	2.63%
GLOVE	29.70%	50.51%	21.15%	32.41%	34.23%	20.74%	2.96%

5 Related Work

Graph-based ANNS. Graph-based approaches [7, 8, 19, 20, 26] are among the most effective methods for ANNS, owing to their strong empirical performance and favorable theoretical properties. Despite their effectiveness, existing graph-based ANNS methods typically rely on fixed termination criteria or globally tuned search parameters, applying the same search budget to all queries, motivating adaptive search and early termination techniques.

Automated Performance Tuning. Recent vector databases and ANN systems expose a large number of tunable parameters that critically affect the trade-off between search efficiency and recall. Several systems explore adaptive tuning of ANN search parameters to reduce manual configuration effort [10, 12]. Recent learning-based approaches further attempt to predict search efficiency, enabling adaptive control of search strategies or parameters [17].

Learning to Early Termination. Recent work explores early termination techniques to reduce search cost by stopping the query once a predefined search objective is satisfied, such as ProS [6, 9] and AdaptNN [16] (ProS is primarily designed for exact similarity search over time series using the iSAX index [3]). More recently, Darth [5] introduces a learned recall predictor to estimate the current recall during graph traversal and triggers termination once a target recall is reached, enabling adaptive early stopping for graph-based ANNS.

6 Conclusion

We propose an early-termination framework in graph-based ANNS that enables query-adaptive control during traversal. It could choose from a dynamic recall predictor to estimate search quality online and a CNS predictor based on quantile regression to conservatively estimate the remaining search budget required to achieve a target recall. Experimental results show that our method achieves substantial reductions in search cost compared to state-of-the-art early termination techniques, without sacrificing recall.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (62372034).

References

[1] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems* 87 (2020), 101374.

[2] Artem Babenko and Victor Lempitsky. 2016. Efficient indexing of billion-scale datasets of deep descriptors. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2055–2063.

[3] Alessandro Camerra, Themis Palpanas, Jin Shieh, and Eamonn Keogh. 2010. isax 2.0: Indexing and mining one billion time series. In *2010 IEEE international conference on data mining*. IEEE, 58–67.

[4] Matteo Ceccarello, Alexandra Levchenko, Ioana Ileana, and Themis Palpanas. 2025. Evaluating and generating query workloads for high dimensional vector similarity search. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 5299–5310.

[5] Manos Chatzakis, Yannis Papakonstantinou, and Themis Palpanas. 2025. DARTH: Declarative Recall Through Early Termination for Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 3, 4 (2025), 1–26.

[6] Karima Echihabi, Theophanis Tsandilas, Anna Gogolou, Anastasia Bezerianos, and Themis Palpanas. 2023. ProS: data series progressive k-NN similarity search and classification with probabilistic quality guarantees. *The VLDB Journal* 32, 4 (2023), 763–789.

[7] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proceedings of the VLDB Endowment* 12, 5 (2019), 461–474.

[8] Cong Fu, Chao Xiang, Chao Wang, and Deng Cai. 2021. NSSG: A Navigable Small World Graph for Optimization of Approximate Nearest Neighbor Search. In *Proceedings of the 37th IEEE International Conference on Data Engineering (ICDE)*. 1997–2002.

[9] Anna Gogolou, Theophanis Tsandilas, Themis Palpanas, and Anastasia Bezerianos. 2019. Progressive similarity search on time series data. In *BigVis 2019-2nd International Workshop on Big Data Visual Exploration and Analytics*.

[10] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. Diskann: Fast accurate billion-point nearest neighbor search on a single node. *Advances in neural information processing Systems* 32 (2019).

[11] Hervé Jégou, Romain Tavenard, Matthijs Douze, and Laurent Amsaleg. 2011. Searching in one billion vectors: re-rank with source coding. In *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 861–864.

[12] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.

[13] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems* 30 (2017).

[14] Atsutake Kosuge and Takashi Oshima. 2019. An object-pose estimation acceleration technique for picking robot applications by using graph-reusing k-nn search. In *2019 First International Conference on Graph Computing (GC)*. IEEE, 68–74.

[15] Brian Kulis and Kristen Grauman. 2009. Kernelized locality-sensitive hashing for scalable image search. In *2009 IEEE 12th international conference on computer vision*. IEEE, 2130–2137.

[16] Conglong Li, Minjia Zhang, David G Andersen, and Yuxiong He. 2020. Improving approximate nearest neighbor search through learned adaptive early termination. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 2539–2554.

[17] Mingjie Li, Yuan-Gen Wang, Peng Zhang, Hanpin Wang, Lisheng Fan, Enxia Li, and Wei Wang. 2022. Deep learning for approximate nearest neighbour search: A survey and future directions. *IEEE Transactions on Knowledge and Data Engineering* 35, 9 (2022), 8997–9018.

[18] Qin Lv, Moses Charikar, and Kai Li. 2004. Image similarity search with compact data structures. In *Proceedings of the thirteenth ACM international conference on Information and knowledge management*. 208–217.

[19] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* 45 (2014), 61–68.

[20] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.

[21] Yitong Meng, Xinyan Dai, Xiao Yan, James Cheng, Weiwen Liu, Jun Guo, Benben Liao, and Guangyong Chen. 2020. Pmd: An optimal transportation-based user distance for recommender systems. In *Advances in Information Retrieval: 42nd European Conference on IR Research, ECIR 2020, Lisbon, Portugal, April 14–17, 2020, Proceedings, Part II 42*. Springer, 272–280.

[22] Jeffrey Pennington, Richard Socher, and Christopher D Manning. 2014. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*. 1532–1543.

[23] James Philbin, Ondrej Chum, Michael Isard, Josef Sivic, and Andrew Zisserman. 2007. Object retrieval with large vocabularies and fast spatial matching. In *2007 IEEE conference on computer vision and pattern recognition*. IEEE, 1–8.

[24] Nick Roussopoulos, Stephen Kelley, and Frederic Vincent. 1995. Nearest neighbor queries. In *Proceedings of the 1995 ACM SIGMOD international conference on Management of data*. 71–79.

[25] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamny, Gopal Srinivasa, et al. 2022. Results of the NeurIPS'21 challenge on billion-scale approximate nearest neighbor search. In *NeurIPS 2021 Competitions and Demonstrations Track*. PMLR, 177–189.

[26] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *Proceedings of the VLDB Endowment* 14, 11 (2021), 1964–1978.

[27] Zeyu Wang, Qitong Wang, Xiaoxing Cheng, Peng Wang, Themis Palpanas, and Wei Wang. 2024. Steiner-hardness: A query hardness measure for graph-based ann indexes. *Proceedings of the VLDB Endowment* 17, 13 (2024), 4668–4682.

[28] Roger Weber, Hans-Jörg Schek, and Stephen Blott. 1998. A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces. In *VLDB*, Vol. 98. 194–205.

[29] Kostas Zoumpatianos, Yin Lou, Themis Palpanas, and Johannes Gehrke. 2015. Query workloads for data series indexes. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 1603–1612.

A Pseudocode for Two Predictors

A.1 Pseudocode for Recall Predictor

Algorithm 1 describes the early termination procedure based on recall prediction. The algorithm starts by computing the distance between the entry node p and the query q, and inserts p into the CNS Q. Then it initializes the variables to track the stability duration, predicted recall, and upper recall bound, which are used to control when recall prediction is triggered (lines 1-3). The search proceeds by iteratively selecting the first unvisited node from Q and expanding its neighbors. For each neighbor n, the algorithm computes its distance to the query and inserts it into Q if it improves the candidate set (lines 7–9). If n enters the top-k positions, the recall upper bound is incremented by $1/k$, and the number of top-k insertions in the current iteration is recorded (lines 10–12). After each expansion, the algorithm checks whether the candidate set has remained stable for at least μ_γ iterations. Once this condition is met, the recall predictor is invoked either for the first time, or when the recall upper bound already exceeds the target recall, or when the stability duration reaches μ_1, which serves as a safeguard against systematic underestimation (lines 13–18). If the predicted recall satisfies the target recall γ, the algorithm terminates early and returns the top-k nodes in Q. Otherwise, the predicted recall is used to update the recall upper bound, constraining future prediction attempts. At the end of each iteration, the algorithm updates the stability duration based on the number of new insertions into the top-k positions (lines 19-22). If no early termination condition is met, the algorithm continues until all reachable candidates are explored and returns the final top-k results.

A.2 Pseudocode for CNS Predictor

Algorithm 2 illustrates the early termination strategy based on CNS prediction, which estimates the remaining candidate set size required to achieve the target recall. The algorithm begins by computing the distance between the entry node p and the query q, inserting p into the CNS Q as the initial candidate (lines 1-2). The search proceeds in a standard graph-based manner. At each iteration, the first unvisited node in Q is selected and its unvisited neighbors are explored (lines 4-7). Once the candidate set reaches

Algorithm 1: Early Termination with Recall Prediction

Input: graph G, query q, result set size k, sorted candidate set Q (ascending), candidate set size L, target recall γ, recall predictor Φ_r, average stable duration μ_r and μ_1 for the training set to reach γ and 1, respectively

Output: the k nearest neighbors of q

```
1  Compute the distance d⟨p, q⟩ from the entry node p to q;
2  Q.add(p, d⟨p, q⟩);
3  stable_duration, predict_recall, recall_bound ← 0;
4  while unvisited nodes exist in Q do
5      p ← the first unvisited node in Q ;
6      insertion ← 0;
7      for each unvisited neighbor n of p do
8          if Q.size() < L or d⟨n, q⟩ < Q[L − 1].distance then
9              Q.add(n, d⟨n, q⟩);
10             if d⟨n, q⟩ < Q[k − 1].distance then
11                 recall_bound←recall_bound+1/k;
12                 insertion ← insertion + 1;
13         if stable_duration ≥ μ_γ then
14             if this is the first prediction or recall_bound ≥ γ
                 or stable_duration==μ_1 then
15                 predict_recall ← Φ_r.predict();
16                 if predict_recall ≥ γ then
17                     return the first k nodes in Q
18                 recall_bound←predict_recall;
19     if insertion ≤ k − ⌈k × γ⌉ then
20         stable_duration ← stable_duration + 1;
21     else
22         stable_duration ← 0;
23 return the first k nodes in Q
```

the k-th position, the recall predictor is invoked to estimate the minimal candidate set size required to achieve the target recall. The candidate set size L is then updated to the maximum of the predicted value and k, ensuring correctness, and Q is resized accordingly to reflect the predicted search budget (lines 8-10). The search continues until no unvisited nodes remain in Q.

B Additional Experimental Results

B.1 Details of Feature Importance

Table 11 provides a fine-grained view of feature importance for recall prediction. Across all datasets and both graph indices, distance features consistently dominate, with the mean and variance (μ_d and σ_d) of distances and the distance to the current nearest neighbor (d_0) being the most influential. This indicates that recall is primarily determined by how tightly the current CNS clusters around the query. Search progress features also contribute substantially, reflecting the monotonic nature of recall growth as the search proceeds. In contrast, CNS stability features have relatively small importance under HNSW but become noticeably more relevant

Algorithm 2: Early Termination with CNS Prediction

Input: graph G, query q, result set size k, sorted candidate set Q (ascending), candidate set size L, target recall γ, cns predictor Φ_c

Output: the k nearest neighbors of q

```
1  Compute the distance d⟨p, q⟩ from the entry node p to q;
2  Q.add(p, d⟨p, q⟩);
3  while unvisited nodes exist in Q do
4      p ← the first unvisited node in Q ;
5      for each unvisited neighbor n of p do
6          if Q.size() < L or d⟨n, q⟩ < Q[L − 1].distance then
7              Q.add(n, d⟨n, q⟩);
8      if it is the kth position of Q then
9          L ← max(Φ_c.predict(), k);
10         Q.resize(L);
11 return the first k nodes in Q
```

Table 11: Feature Importance of Recall Predictor

Dataset	n_{step}	n_{dis}	n_{insert}	d_0	d_{k-1}	μ_d	σ_d	r_{visited}	n_{stable}
SIFT	11.90%	11.90%	10.83%	20.46%	0.70%	16.93%	22.00%	0.20%	5.06%
GIST	10.76%	9.96%	9.73%	23.16%	1.20%	17.10%	22.46%	0.00%	5.60%
DEEP	8.83%	15.90%	11.56%	5.73%	3.30%	28.23%	20.73%	0.33%	5.36%
GLOVE	18.33%	14.56%	10.33%	14.53%	7.93%	13.50%	15.56%	1.60%	3.63%

(a) HNSW

Dataset	n_{step}	n_{dis}	n_{insert}	d_0	d_{k-1}	μ_d	σ_d	r_{visited}	n_{stable}
SIFT	8.70%	7.88%	3.68%	22.23%	6.35%	18.68%	17.93%	11.10%	3.28%
GIST	15.42%	11.93%	9.23%	11.90%	10.92%	7.00%	16.23%	11.18%	6.18%
DEEP	16.08%	6.75%	6.73%	5.6%	7.53%	22.92%	16.25%	13.45%	4.68%
GLOVE	7.68%	10.47%	5.58%	13.10%	9.60%	15.60%	20.77%	12.70%	4.50%

(b) NSG

under NSG, especially the ratio of visited nodes among the first k potision (r_{visited}).

As shown in Table 12, the importance distribution for CNS prediction is significantly more dispersed. No single feature dominates and the prediction relies on a combination of heterogeneous signals. Search progress features, especially the average hop μ_{hop}, play a critical role, highlighting that the required CNS size is closely tied to query-specific search difficulty. Distance features further capture how broadly the current candidates are distributed. The distance entropy H_d captures the dispersion of candidates in the CNS and provides information beyond mean and variance in some cases. The features related to the entry node is also important, especially under HNSW. Cosine similarity features contribute prominently on high-dimensional datasets such as GIST, indicating that directional consistency provides complementary information beyond distance when estimating the remaining exploration budget. Degree features under NSG exhibit lower but non-negligible importance, capturing additional structural properties of the graph.

B.2 Ablation Study of Recall Predictor

We remove three components of the recall predictor Φ_r one at a time, and Table 13 reports the corresponding search time and the average recall achieved (shown in parentheses). Specifically, "w/o

Table 12: Feature Importance of CNS Predictor

Dataset	n_{step}	n_{dis}	n_{insert}	μ_{hop}	d_0	d_{k-1}	μ_d	σ_d	H_d	d_{en}	r_{en}	μ_{cos}	σ_{cos}	H_{cos}
SIFT	2.44%	5.08%	8.22%	13.21%	11.37%	5.06%	5.08%	5.59%	6.76%	7.05%	7.43%	5.10%	2.16%	2.28%
GIST	2.20%	13.63%	5.20%	15.02%	7.03%	4.70%	6.16%	4.54%	5.52%	3.87%	6.91%	17.45%	5.43%	2.07%
DEEP	0.99%	20.09%	6.19%	18,29%	7.05%	3.66%	10.56%	2.69%	6.62%	5.56%	6.19%	2.79%	1.39%	2.66%
GLOVE	1.21%	17.49%	5.93%	5.07%	6.91%	7.90%	7.29%	6.36%	4.49%	5.51%	12.05%	10.21%	4.78%	6.16%

(a) HNSW

Dataset	n_{step}	n_{dis}	n_{insert}	μ_{hop}	d_0	d_{k-1}	μ_d	σ_d	H_d	d_{en}	r_{en}	μ_{cos}	σ_{cos}	H_{cos}	μ_{deg^-}
SIFT	13.50%	4.40%	11.43%	19.43%	13.33%	4.12%	8.50%	2.13%	5.03%	3.90%	3.75%	2.48%	0.79%	0.46%	5.15%
GIST	13.26%	4.86%	10.12%	15.64%	5.12%	3.55%	0.90%	2.72%	3.35%	4.25%	3.42%	14.62%	6.60%	2.11%	3.47%
DEEP	21.65%	1.57%	7.23%	15.57%	0.90%	4.02%	7.93%	3.13%	6.32%	3.61%	2.90%	4.22%	0.26%	1.20%	2.63%
GLOVE	14.52%	8.30%	5.12%	4.47%	6.65%	3.80%	2.00%	12.17%	3.26%	3.32%	3.03%	10.06%	5.82%	4.86%	2.96%

(b) NSG

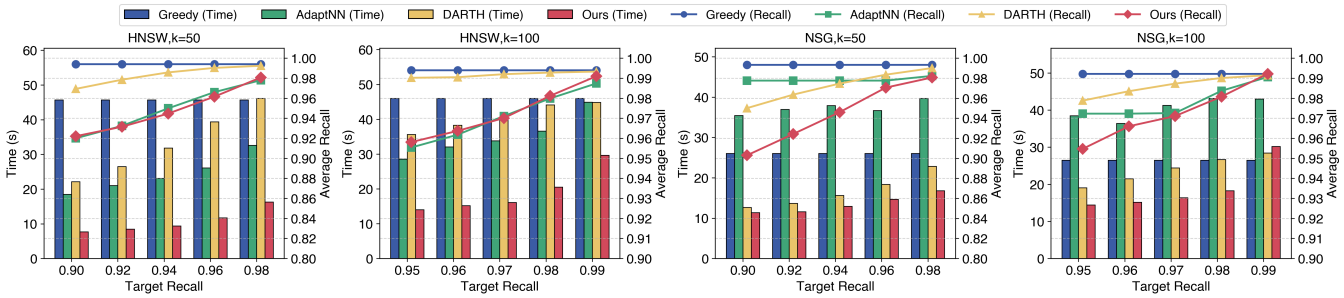


Figure 4: Performance on OOD Workloads

Table 13: Ablation Study of Recall Predictor

Dataset	Φ_r	w/o μ_γ	w/o *UB*	w/o μ_1
SIFT	3.86s (0.944)	4.45s (0.941)	4.07s (0.933)	4.88s (0.945)
GIST	3.02s (0.952)	3.40s (0.950)	3.03s (0.941)	3.78s (0.954)
DEEP	6.13s (0.952)	6.90s (0.948)	6.36s (0.943)	9.51s (0.954)
GLOVE	1.69s (0.940)	2.07s (0.928)	4.28s (0.936)	1.76s (0.940)

(a) HNSW, $k = 50, \gamma = 0.90$

Dataset	Φ_r	w/o μ_γ	w/o *UB*	w/o μ_1
SIFT	7.72s (0.975)	8.67s (0.973)	8.57s (0.969)	9.35s (0.976)
GIST	5.38s (0.976)	5.87s (0.976)	6.08s (0.972)	6.30s (0.978)
DEEP	15.17s (0.981)	16.27s (0.978)	17.37s (0.977)	19.00s (0.982)
GLOVE	2.92s (0.965)	3.41s (0.966)	6.51s (0.961)	3.04s (0.965)

(b) HNSW, $k = 100, \gamma = 0.95$

Dataset	Φ_r	w/o μ_γ	w/o *UB*	w/o μ_1
SIFT	4.57s (0.939)	5.20s (0.924)	4.77s (0.928)	4.97s (0.940)
GIST	4.47s (0.949)	4.77s (0.938)	4.84s (0.934)	5.58s (0.950)
DEEP	6.32s (0.953)	6.65s (0.919)	6.50s (0.944)	7.67s (0.956)
GLOVE	1.77s (0.934)	2.37s (0.914)	1.91s (0.931)	1.93s (0.935)

(c) NSG, $k = 50, \gamma = 0.90$

Dataset	Φ_r	w/o μ_γ	w/o *UB*	w/o μ_1
SIFT	5.41s (0.970)	6.31s (0.960)	5.59s (0.965)	5.46s (0.970)
GIST	7.06s (0.973)	7.21s (0.965)	8.48s (0.968)	7.28s (0.975)
DEEP	13.18s (0.979)	13.58s (0.961)	13.40s (0.975)	13.52s (0.981)
GLOVE	1.94s (0.962)	2.79s (0.957)	2.01s (0.958)	1.96s (0.962)

(d) NSG, $k = 100, \gamma = 0.95$

μ_γ" indicates that a prediction is made as long as the number of neighbors of the visited node inserted into the first k positions of the CNS is below $k - \lceil k \times \gamma \rceil$; "w/o *UB*" indicates that a prediction is triggered whenever a stable state of duration μ_γ is re-entered, without calculating an upper bound (*UB*); "w/o μ_1" indicates that this safeguard mechanism is removed.

The experimental results show that removing μ_γ and *UB* significantly increases the number of predictions. Although this may enable earlier stopping, it also incurs substantially higher prediction overhead. Removing the safeguard μ_1 prevents queries that are consistently underestimated from successfully terminating early, thereby increasing the search time. We also observe that the effectiveness of the safeguard varies across different datasets.

B.3 Comparisons for OOD Workloads

We study the performance of different ANNS methods for the T2I100M dataset [25], which contains out-of-distribution (OOD) queries. We follow the same processing workflow as for other datasets. Figure 4 shows that the average recall of our method consistently meets and exceeds all recall targets and requires less time than the baselines, especially on HNSW, demonstrating its effectiveness in handling OOD queries. We outperform DARTH mainly because its heuristic prediction interval leads to a large number of predictions (19.7 on average on this dataset), which introduces significant overhead. In contrast, our design substantially reduces the number of predictions and enables effective early stopping, leading to improved efficiency. Moreover, the consistent gains across different settings suggest that our approach generalizes well, highlighting its advantage over heuristic-based methods.